

19.图书系统项目（三）

1. book模块

1.1 Book类的定义

关于书籍类我们书籍的属性如下：

private int bookId;	//书id
private String title;	// 书名
private String author;	// 作者
private String category;	// 类别
private int publishYear;	// 出版年份
private boolean isBorrowed;	// 借阅状态
private int borrowCount;	// 借阅次数
private LocalDate shelfDate;	// 上架时间

注意构造方法的实现对于某些字段可以不加入参数：

1. bookId 【这里后续会进行累加/自增操作，此时在构造Book对象的时候，并不知道当前ID】
2. isBorrowed 【任何一种书籍，默认都是未借出状态，此时可以不进行处理】
3. borrowCount 【借阅次数默认为0，此时可以不进行处理】

```
1 public class Book implements Comparable<Book>{
2     private int bookId;           //书id
3     private String title;         // 书名
4     private String author;       // 作者
5     private String category;     // 类别
6     private int publishYear;     // 出版年份
7     private boolean isBorrowed;  // 借阅状态
8     private int borrowCount;     // 借阅次数
9     private LocalDate shelfDate; // 上架时间
10 }
```

```
11 // 构造函数，初始化图书对象 书籍ID、借阅状态和 借阅次数不用进行参数传递
12 public Book(String title, String author, String category,
13             int publishYear, LocalDate shelfDate) {
14     this.title = title;
15     this.author = author;
16     this.category = category;
17     this.publishYear = publishYear;
18     this.isBorrowed = false;
19     this.borrowCount = 0;
20     this.shelfDate = shelfDate;
21 }
22
23 public int getBookId() {
24     return bookId;
25 }
26
27 public void setBookId(int bookId) {
28     this.bookId = bookId;
29 }
30
31 public String getTitle() {
32     return title;
33 }
34
35 public void setTitle(String title) {
36     this.title = title;
37 }
38
39 public String getAuthor() {
40     return author;
41 }
42
43 public void setAuthor(String author) {
44     this.author = author;
45 }
46
47 public String getCategory() {
48     return category;
49 }
50
51 public void setCategory(String category) {
52     this.category = category;
53 }
54
55 public int getPublishYear() {
56     return publishYear;
57 }
```

```
58
59     public void setPublishYear(int publishYear) {
60         this.publishYear = publishYear;
61     }
62
63     public boolean isBorrowed() {
64         return isBorrowed;
65     }
66
67     public void setBorrowed(boolean borrowed) {
68         isBorrowed = borrowed;
69     }
70
71     public int getBorrowCount() {
72         return borrowCount;
73     }
74
75     public void setBorrowCount(int borrowCount) {
76         this.borrowCount = borrowCount;
77     }
78
79     public void incrementBorrowCount() {
80         this.borrowCount++;
81     }
82
83     public void decreaseBorrowCount() {
84         this.borrowCount--;
85     }
86
87     public LocalDate getShelfDate() {
88         return shelfDate;
89     }
90
91     public void setShelfDate(LocalDate shelfDate) {
92         this.shelfDate = shelfDate;
93     }
94
95     @Override
96     public String toString() {
97         return "Book{" +
98             "bookId='" + bookId + '\'' +
99             ",title='" + title + '\'' +
100             ", author='" + author + '\'' +
101             ", category='" + category + '\'' +
102             ", publishYear=" + publishYear +
103             ", isBorrowed=" + isBorrowed +
104             ", borrowCount=" + borrowCount +
```

```

105         ", shelfDate=" + shelfDate +
106         '}}';
107     }
108
109     @Override
110     public int compareTo(Book o) {
111         return o.getBorrowCount() - this.getBorrowCount();
112     }
113 }

```

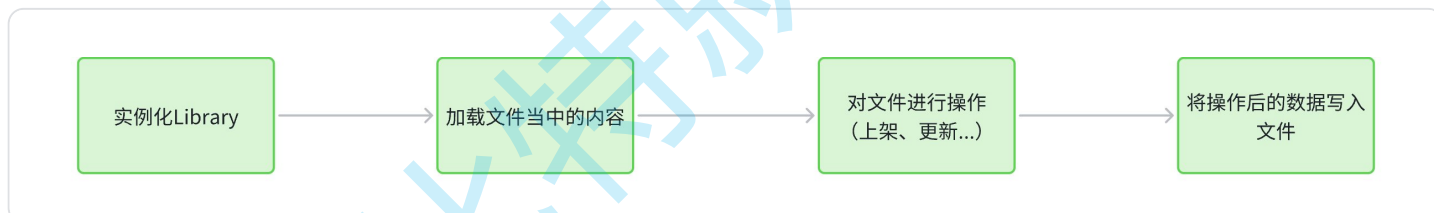
这里我们默认实现了 `Comparable` 接口，对后期对象的比较做一个准备工作。

1.2 Library类的定义

1.2.1 Library中的设计思路

我们期望书籍等数据最好可以进行持久化，所谓持久化就是将数据存储到MySQL、文件等介质中，不会因为断电等情况影响数据。否则每次运行程序数据都需要重写进行存储等比较繁琐。由于目前知识储备，我们还没有学习到MySQL和文件，我们这里会采用文件进行存储，把文件相关操作封装到 `jar` 包，借助已经写好的工具来进行操作。我们这里先根据《17.图书系统项目（一）》中把给定的`jar`包进行导入项目。接下来我们来看整体的设计流程。

实例化 `Library` -> 加载文件当中的内容-> 对文件进行操作-> 将操作后的数据写入文件



1.2.2 AnalyzingBook 类实现文件内容的写入

在util包下创建 `AnalyzingBook` 类：实现的功能如下

```

1  public class AnalyzingBook {
2
3      public void storeObject(Book[] books, String filename) throws IOException
4      {
5      }
6
7      public Book[] loadObject(String filename) throws IOException {
8          //从文件读取数据
9
10     }
11 }

```

```
12
13     private Book parseBookJson(String json) {
14
15     }
16 }
```

现有一组书籍进行文件的存储

```
1  Book[] books = new Book[4];
2  books[0] = new Book("java", "gaobo", "编程", 1994, LocalDate.of(2023, 9, 24));
3  books[1] = new Book("mysql", "lisi", "编程", 1999, LocalDate.of(2024, 2, 10));
4  books[2] = new Book("php", "gaobo", "编程", 2020, LocalDate.of(2023, 9, 23));
5  books[3] = new Book("西游记", "吴承恩", "小说", 2024, LocalDate.of(2023, 9, 23));
```

当存入文件之后，文件一行代表一本书籍的书籍对象。写入书籍思路：

1. 实现 `AnalyzingBook` 类
2. 先将书籍数组对象中的每个书籍对象进行字符串的序列化。
3. 每本书籍与书籍之间使用 `\n` 进行换行分割

`books` 代表书籍数组对象

`filename` 代表文件名称【如果不存在该文件，会自动创建该文件】

```
1  public void storeObject(Book[] books, String filename) throws IOException {
2      //先遍历books数组当中不为空的数据多少个?
3      int booksUseLen = 0;
4      for (int i = 0; i < books.length; i++) {
5          if(books[i] != null) {
6              booksUseLen++;
7          }
8      }
9      StringBuilder jsonArray = new StringBuilder();
10     for (int i = 0; i < booksUseLen; i++) {
11         if(books[i] != null) {
12             jsonArray.append(books[i].toJSON());
13             if (i != booksUseLen-1) {
14                 //一本书籍完成后以\n进行分割
15                 jsonArray.append("\n");
16             }
17         }
18     }
```

```
19 //数据写入文件
20 FileUtils.writeFile(jsonArray.toString(),filename);
21 }
```

注意：

- 上述代码当中toJSON需要我们自己实现
- FileUtils中的writeFile方法已经实现好，可以直接使用

1.2.3 Book类中toJson的实现

```
1, java, gaobo, 编程, 1994, true, 1, 2023-09-24
2, mysql, lisi, 编程, 1999, false, 0, 2024-02-10
3, PHP, gaobo, 编程, 2020, true, 1, 2023-09-23
4, 西游记, 吴承恩, 小说, 2024, false, 0, 2023-09-23
5, 图书测试, author, 测试类别, 2024, false, 0, 2024-10-10
```

在Book类当中实现 `toJson` 方法，实现逻辑是将每个属性使用逗号进行拼接，存储到文件即可

```
1 public String toJson() {
2     StringBuilder json = new StringBuilder();
3     json.append(bookId).append(",");
4     json.append(title).append(",");
5     json.append(author).append(",");
6     json.append(category).append(",");
7     json.append(publishYear).append(",");
8     json.append(isBorrowed).append(",");
9     json.append(borrowCount).append(",");
10    json.append(shelfDate != null ?
        shelfDate.format(DateTimeFormatter.ISO_LOCAL_DATE) : "null");
11    return json.toString();
12 }
```

1.2.4 AnalyzingBook 类实现文件内容的读取

读取文件的每一行字符串，转化为书籍对象。

1. 先读取文件当中的所有内容
2. 使用 `\n` 作为分隔符进行字符串分割，方便得到每一个书籍对象对应的字符串
3. 把对应字符串“组装”成书籍对象

```

1  public Book[] loadObject(String filename) throws IOException {
2      //1. 从文件读取数据
3      String content = FileUtils.readFile(filename);
4      if (content == null || content.isEmpty()) {
5          System.out.println("File is empty or does not exist: " + filename);
6          return null;
7      }
8      //2. 使用\n作为分隔符进行字符串分割
9      String[] bookJsonStrings = content.split("\n");
10
11     //3. 把对应字符串“组装”成书籍对象
12     Book[] bookList = new Book[bookJsonStrings.length];
13     for (int i = 0; i < bookJsonStrings.length; i++) {
14         Book book = parseBookJson(bookJsonStrings[i]);
15         bookList[i] = book;
16     }
17
18     return bookList;
19 }

```

parseBookJson 方法实现

1. 检查json是否是空的
2. 使用逗号分割出每个属性的值
3. 每个属性进行转化赋值
4. 构造书籍对象

```

1  private Book parseBookJson(String json) {
2      //1. 检查json是否是空的
3      if(json.isEmpty()) {
4          return null;
5      }
6      //2. 使用逗号分割出每个属性的值
7      String[] pairs = json.split(",");
8      //3. 每个属性进行转化赋值
9      int bookId = Integer.parseInt(pairs[0]);
10     String title = pairs[1];
11     String author = pairs[2];
12     String category = pairs[3];
13     int publishYear = Integer.parseInt(pairs[4]);
14     boolean isBorrowed = Boolean.parseBoolean(pairs[5]);
15     int borrowCount = Integer.parseInt(pairs[6]);
16     LocalDate shelfDate = LocalDate.parse(pairs[7]);
17     //4. 构造书籍对象

```

```

18     if (title != null && author != null && category != null && shelfDate !=
19         null) {
20         Book book = new Book(title, author, category, publishYear, shelfDate);
21         book.setBorrowed(isBorrowed);
22         book.setBorrowCount(borrowCount);
23         book.setBookId(bookId);
24         return book;
25     }
26     return null;

```

测试Test

```

1  public static void main(String[] args) throws IOException {
2      AnalyzingBook myProperties = new AnalyzingBook();
3
4      // 存储数据
5      Book[] books = new Book[4];
6      books[0] = new Book("java", "gaobo", "编程", 1994, LocalDate.of(2023, 9,
7          24));
8      books[1] = new Book("mysql", "lisi", "编程", 1999, LocalDate.of(2024, 2,
9          10));
10     books[2] = new Book("php", "gaobo", "编程", 2020, LocalDate.of(2023, 9,
11         23));
12     books[3] = new Book("西游记", "吴承恩", "小说", 2024, LocalDate.of(2023,
13         9, 23));
14     myProperties.storeObject(books, Constant.ALL_BOOK_FILE_NAME);
15
16     // 读取数据
17     Book[] loadedBooks =
18     myProperties.loadObject(Constant.ALL_BOOK_FILE_NAME);
19     if (loadedBooks != null) {
20         System.out.println("Loaded books:");
21         for (Book book : loadedBooks) {
22             System.out.println(book);
23         }
24     }
25 }

```

1.2.5 Library文件&读取存储具体细节设计

1.2.5.1 文件内容读取到内存中进行存储

文件



读取文件解析



存储到内存当中

既然我们要把解析的数据存储到内存当中，那么我们需要在内存当中定义一个数组来存储读取到的书籍对象。

定义Library类

```
1 public class Library {
2     private Book[] books; //当前图书数组
3     private int bookCount; //实际存储的图书数量
4
5     public Library() {
6         //当调用该构造方法的时候，要加载文件当中的数据进行到books数组当中
7     }
8 }
```

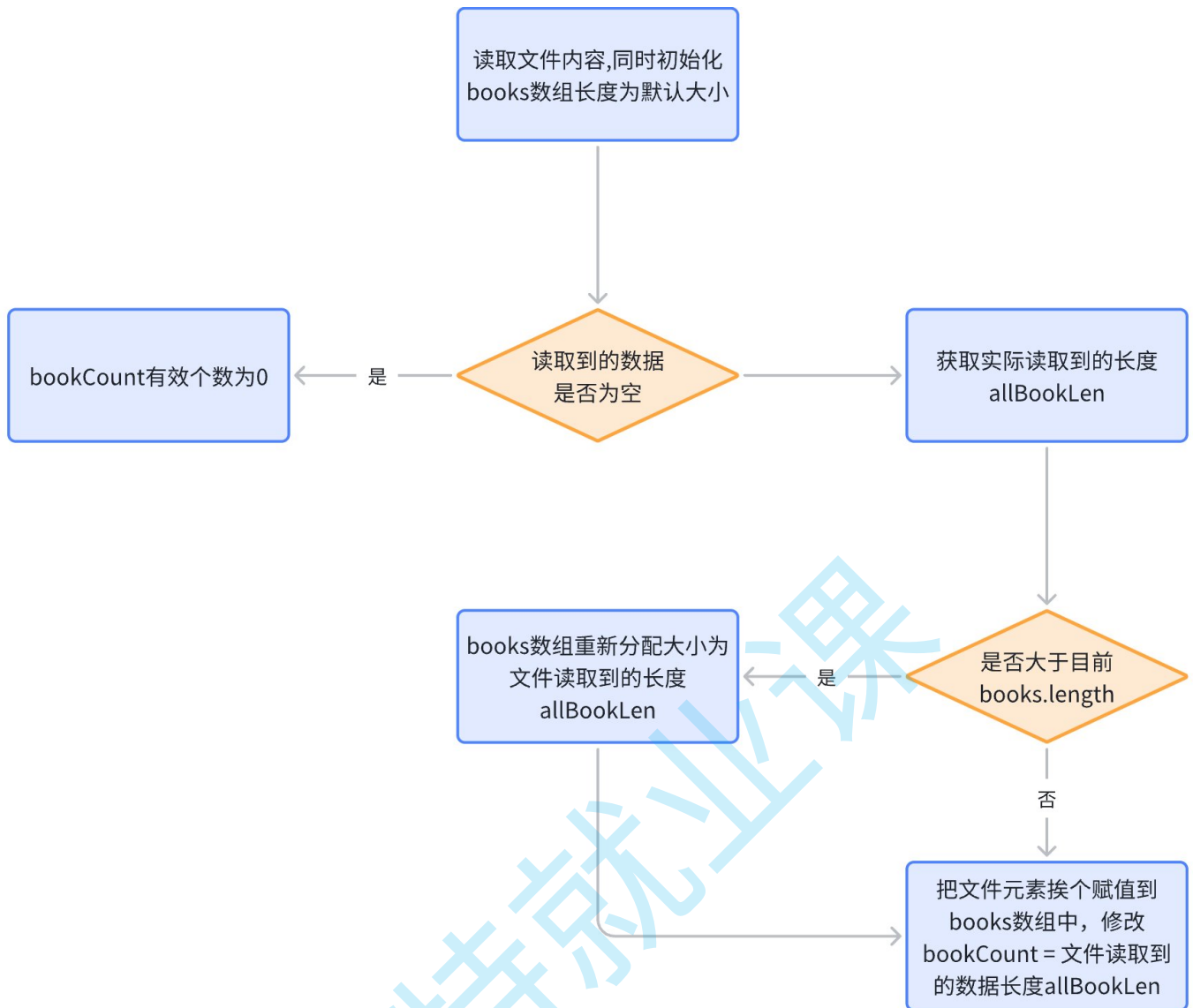
那么问题来了，books数组初始化多大合适呢？我们的思路是：

1. 默认数组容量给一个默认值，比如 5
2. 然后根据实际读取到的文件内容，来确定大小

这个默认值是一个常量，我们在 `constant` 包下定义一个常量类 `Constant` 来存储这些常量

```
1 public class Constant {
2     //内存中的书籍数组初始容量
3     public static final int CAPACITY = 5;
4
5 }
```

实现文件读取内容的存储



代码实现如下:

```
1 private void loadAllBook() {
2     try {
3         //1. 读取文件内容
4         Book[] allBook =
analyzingBook.loadObject(Constant.ALL_BOOK_FILE_NAME);
5         //默认大小为5
6         books = new Book[Constant.CAPACITY];
7         //2. 是否有数据 没有数据 有效书籍个数为 0
8         if(allBook == null) {
9             bookCount = 0;
10        }else {
11            //3. 查看实际书籍长度是多少 是否大于默认的长度5
12            int allBookLen = allBook.length;
13            //3.1 大于默认长度 books数组 分配实际的大小
14            if (allBookLen > books.length) {
```

```

15          //按照实际情况进行分配数组内存
16          books = new Book[allBookLen];
17      }
18      //3.2 把读到的元素进行赋值
19      for (int i = 0; i < allBookLen; i++) {
20          books[i] = allBook[i];
21      }
22      //4. 修改实际有效书籍个数
23      bookCount = allBookLen;
24  }
25  } catch (IOException e) {
26      throw new RuntimeException(e);
27  }
28  }

```

analyzingBook: 为成员变量AnalyzingBook的引用

1.2.5.2 内存中的书籍数组进行存储到文件

```

1  //存储图书到文件中
2  private void storeBook() {
3      try {
4          analyzingBook.storeObject(books, Constant.ALL_BOOK_FILE_NAME);
5      } catch (IOException e) {
6          throw new RuntimeException(e);
7      }
8  }

```

analyzingBook: 为成员变量AnalyzingBook的引用

常量类添加常量: Constant.ALL_BOOK_FILE_NAME

```

1  public class Constant {
2
3      //内存中的书籍数组初识容量
4      public static final int CAPACITY = 5;
5
6      //存储所有图书的文件
7      public static final String ALL_BOOK_FILE_NAME = "allbook.txt";
8  }

```

什么情况下，我们要定义常量？

1. 当有一个值，它在程序运行期间不会改变，并且需要在多个地方使用。
2. 当需要表示某些固定的值，比如常数，错误代码、颜色代码、消息类型等。

1.2.5.3 Library整合文件读取&存储

```
1  public class Library {
2      private Book[] books; //当前图书数组
3      private int bookCount; //实际图书数量
4
5      //书籍解析相关类
6      private final AnalyzingBook analyzingBook = new AnalyzingBook();
7
8      public Library() {
9          loadAllBook();
10     }
11
12     //读取文件
13     private void loadAllBook() {
14         try {
15             //1. 读取文件内容
16             Book[] allBook =
analyzingBook.loadObject(Constant.ALL_BOOK_FILE_NAME);
17             books = new Book[Constant.CAPACITY];
18             //2. 是否有数据 没有数据 有效书籍个数为 0
19             if(allBook == null) {
20                 bookCount = 0;
21             }else {
22                 //3. 查看实际书籍长度是多少 是否大于默认的长度5
23                 int allBookLen = allBook.length;
24                 //3.1 大于默认长度 books数组 分配实际的大小
25                 if (allBookLen > books.length) {
26                     //按照实际情况进行分配数组内存
27                     books = new Book[allBookLen];
28                 }
29                 //3.2 把读到的元素进行赋值
30                 for (int i = 0; i < allBookLen; i++) {
31                     books[i] = allBook[i];
32                 }
33                 //4. 修改实际有效书籍个数
34                 bookCount = allBookLen;
35             }
36         } catch (IOException e) {
37             throw new RuntimeException(e);
38         }
39     }
40     //存储图书到文件中
```

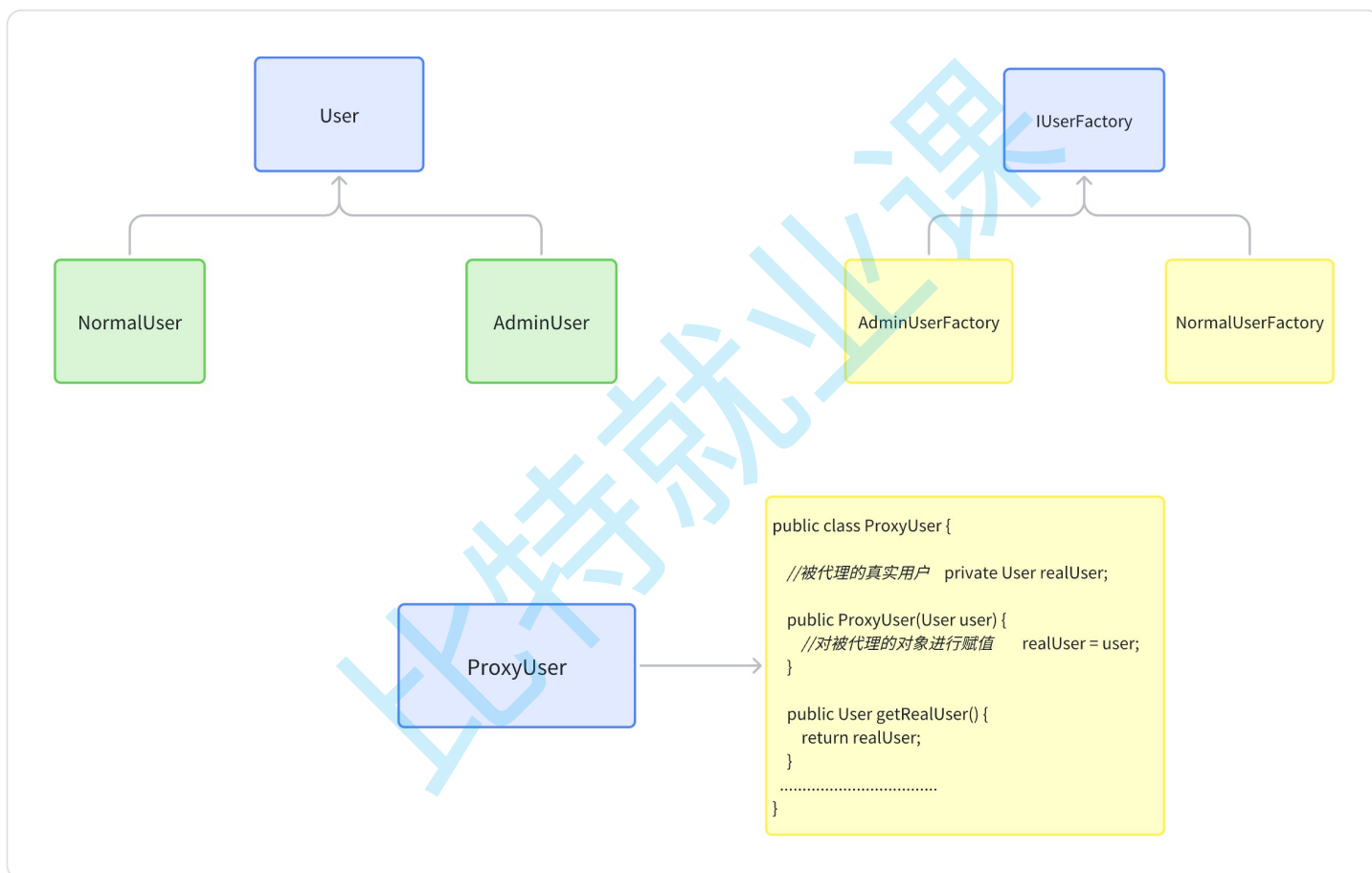
```

41     private void storeBook() {
42         try {
43             analyzingBook.storeObject(books, Constant.ALL_BOOK_FILE_NAME);
44         } catch (IOException e) {
45             throw new RuntimeException(e);
46         }
47     }
48 }

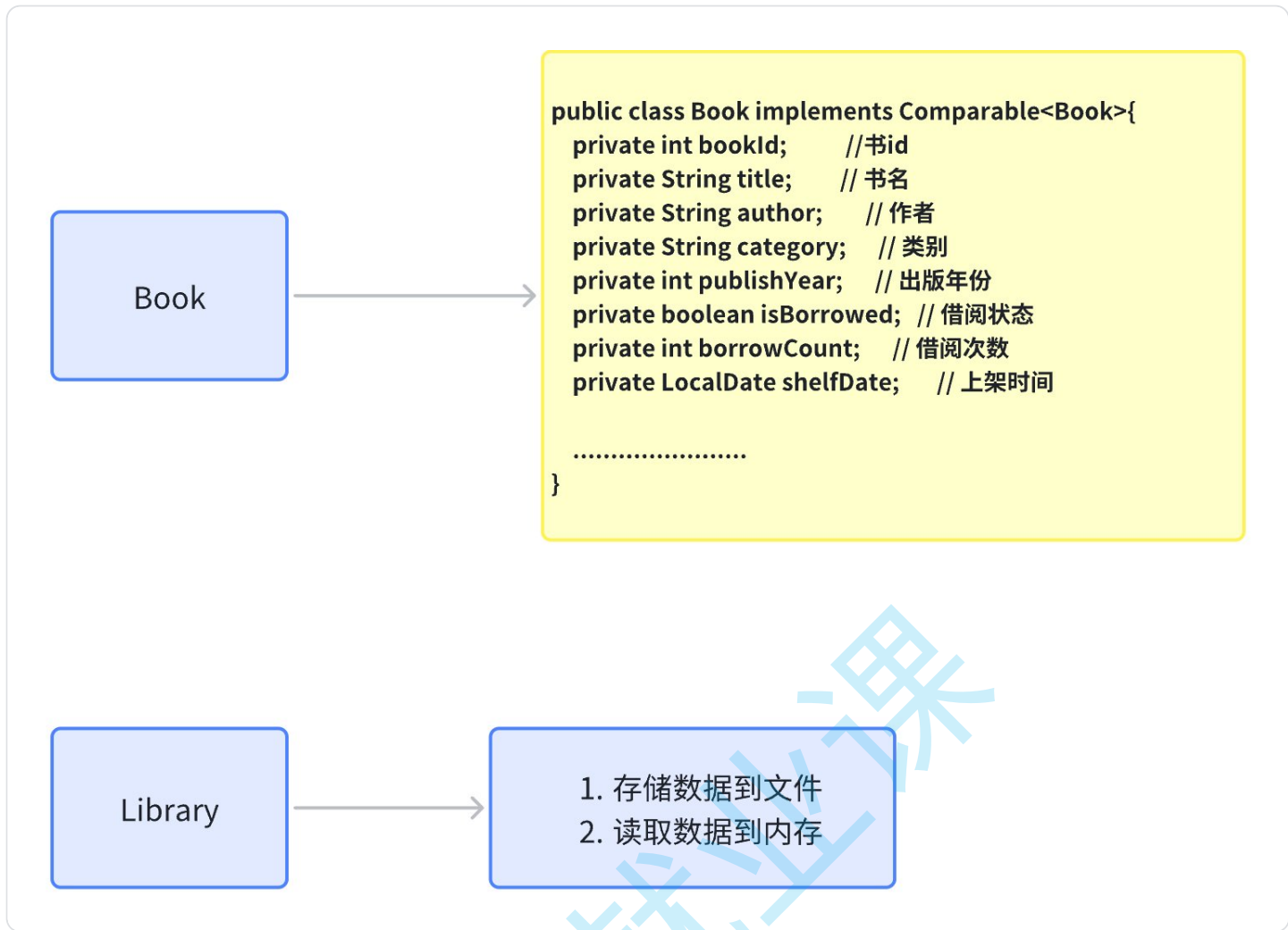
```

1.3 整体回顾当前项目框架

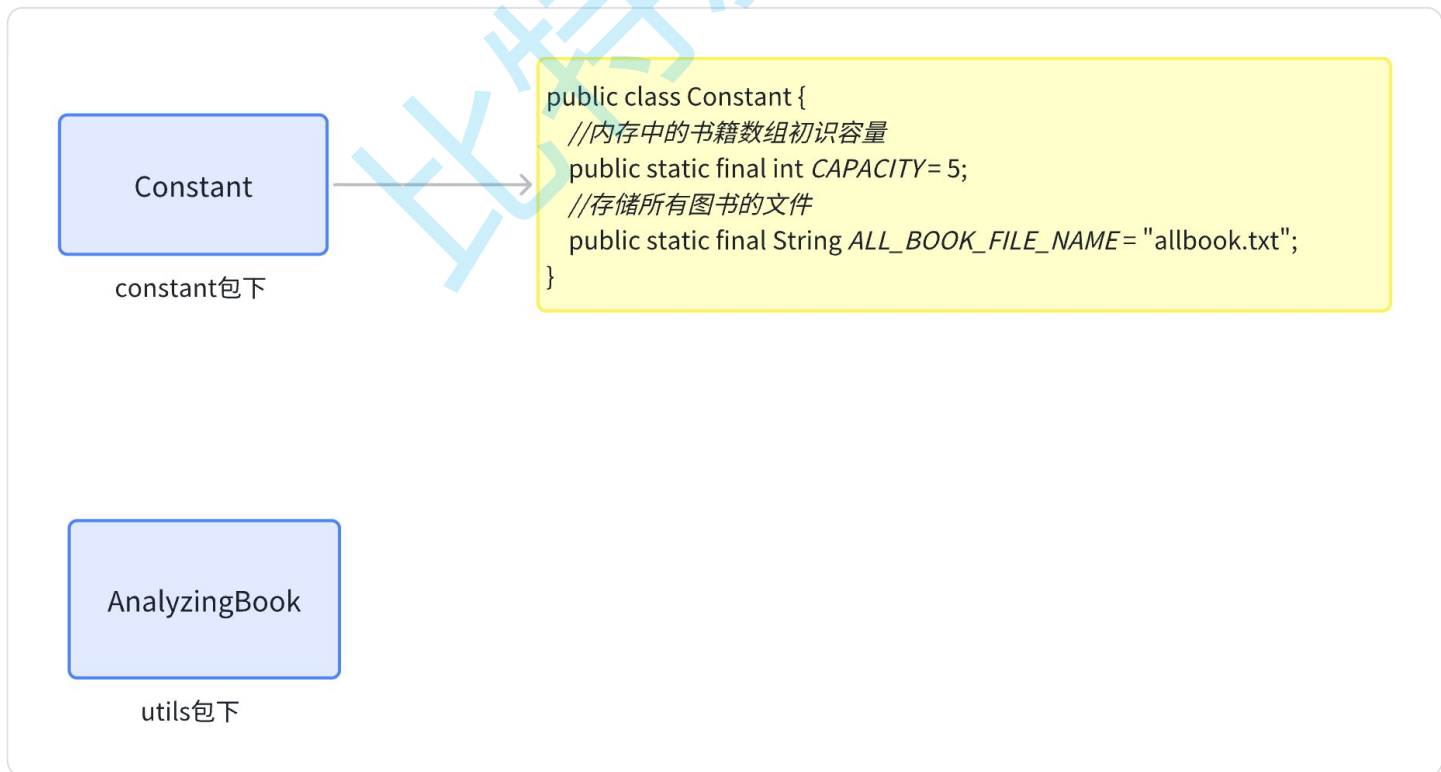
1.3.1 用户相关



1.3.2 书籍相关



1.3.3 其他模块



1.4 业务操作逻辑框架分解

接下来我们需要完成业务逻辑的实现，首先我们需要对框架进行梳理，在正式梳理之前，我们需要完善遗留工作：根据用户不同选择进入不同等级的用户角色下！

1.4.1.1 完善 LibrarySystem 类

在上一节中，我们已经完成了用户逻辑的整理：

LibrarySystem 类中进行整合

```
1  public class LibrarySystem{
2      public static void main(String[] args) {
3
4          IUserFactory adminUserFactory = new AdminUserFactory();
5          User adminUser = adminUserFactory.createUser("刘备",1);
6
7          IUserFactory normalUserFactory = new NormalUserFactory();
8          User normalUser1 = normalUserFactory.createUser("关羽",2);
9          User normalUser2 = normalUserFactory.createUser("张飞",3);
10         /**
11          * 1.4 使用代理模式来管理权限
12          * 使用代理模式来控制 对象的访问
13          */
14
15         ProxyUser proxyUserAdmin = new ProxyUser(adminUser);
16         ProxyUser proxyUserNormalG = new ProxyUser(normalUser1);
17         ProxyUser proxyUserNormalZ = new ProxyUser(normalUser2);
18
19     }
20 }
```

此时的 proxyUserAdmin、proxyUserNormalG、proxyUserNormalZ，是我们的代理对象，我们需要根据用户的选择，来确定最终的角色。

在 LibrarySystem 类中实现 selectProxyRole 方法：

```
1  //选择对应角色进行登录
2  public ProxyUser selectProxyRole(ProxyUser proxyUserAdmin,
3                                  ProxyUser proxyUserNormalW,ProxyUser
4  proxyUserNormalL) {
5      System.out.println("选择角色进行登录：");
6      System.out.println("1.管理员\n2.普通用户(关羽)\n3.普通用户(张飞)\n4.退出系统");
7      ProxyUser currentUser = null;
8
9      Scanner scanner = new Scanner(System.in);
```

```

10     int choice = scanner.nextInt();
11     switch (choice) {
12         case 1:
13             currentUser = proxyUserAdmin;
14             break;
15         case 2:
16             currentUser = proxyUserNormalW;
17             break;
18         case 3:
19             currentUser = proxyUserNormalL;
20             break;
21         case 4:
22             System.exit(0);
23             System.out.println("系统已退出..");
24             break;
25         default:
26             break;
27     }
28     return currentUser;
29 }

```

完善 `LibrarySystem` 类:

```

1  public static void main(String[] args) {
2
3      IUserFactory adminUserFactory = new AdminUserFactory();
4      User adminUser = adminUserFactory.createUser("刘备",1);
5
6      IUserFactory normalUserFactory = new NormalUserFactory();
7      User normalUser1 = normalUserFactory.createUser("关羽",2);
8      User normalUser2 = normalUserFactory.createUser("张飞",3);
9
10     /**
11     * 1.4 使用代理模式来管理权限
12     * 使用代理模式来控制 对象的访问
13     */
14
15     ProxyUser proxyUserAdmin = new ProxyUser(adminUser);
16     ProxyUser proxyUserNormalG = new ProxyUser(normalUser1);
17     ProxyUser proxyUserNormalZ = new ProxyUser(normalUser2);
18
19
20     LibrarySystem librarySystem = new LibrarySystem();
21
22     //选择对应角色进行登录

```



```

23     ProxyUser currentUser =
24
25     librarySystem.selectProxyRole(proxyUserAdmin,proxyUserNormalG,proxyUserNormalZ
26     );
27
28     while (true) {
29
30         int choice = currentUser.getRealUser().display();
31
32         //此时无需关系是 管理员还是普通用户，代理类会做权限判断
33         currentUser.handleOperation(choice);
34     }
35
36 }

```

`handleOperation` 方法会在 `ProxyUser` 类中进行实现

1.4.1.2 业务框架搭建

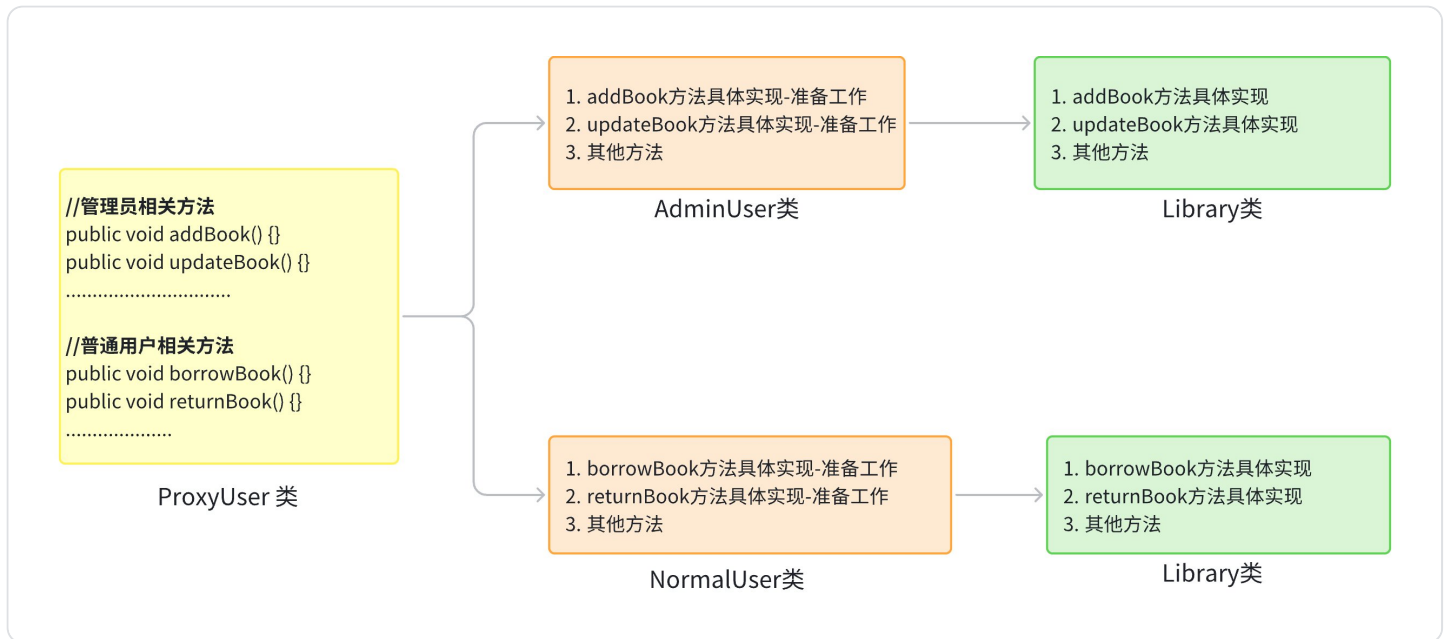
管理员的业务逻辑有

- 查找图书
- 显示图书
- 退出系统
- 上架图书
- 更新图书
- 下架图书
- 查看图书的借阅次数
- 查看最受欢迎的前K本书
- 查看库存状态
- 移除上架超过1年的书籍

普通用户的逻辑有

- 查找图书
- 显示图书
- 退出系统
- 借阅图书
- 归还图书
- 查看个人借阅情况

整个业务逻辑从 `ProxyUser` 类进行管理触发，整个框架为：



完

比特就业课